Math 6601 Independent Study Project

Jacob Hauck

April 23, 2025

Proposal

For my independent study project I would like to study one of the most popular methods for computing all the eigenvalues and eigenvectors of an $n \times n$ matrix A: the QR method [1].

Goals

I would like to achieve the following goals with this project.

1. Understand the theory of the QR method by writing a description of the algorithm and a proof of its convergence based on the discussion in our textbook [1], filling the gaps of the textbook and explaining the rationale and motivation of the method.
2. Create an implementation of the QR method in MATLAB.

Project Tasks

I plan to achieve the above goals through the following steps.

1. Describe background on the QR decomposition, the primary tool used in the QR method: what it is, and how it is computed. I will use our textbook [1] and also the textbook from Applied Matrix Analysis [2] as sources for my own description.
2. Learn how to reduce a matrix to Hessenberg form by using Householder transformations, based on the description in the textbook. I will write a description of the algorithm and implement it in MATLAB. This first step of the QR method reduces the computational complexity of one step of the QR method iteration from $\mathcal{O}(n^3)$ to $\mathcal{O}(n^2)$.
3. I may need to create my own implementation of the QR decomposition for a Hessenberg matrix (I don't know if the built-in MATLAB implementation will do this) in order to exploit the advantage listed above.
4. Describe the QR method and its motivation. Here I will draw on our textbook but fill in gaps by proving assertions that the textbook makes without proof and explaining the motivation of the method as I understand it.
5. Prove the convergence of the QR method. I will write my own version of the textbook's proof, again filling in gaps and expanding on things that the textbook glosses over.
6. Implement the QR method. With the fast QR decomposition and reduction to Hessenberg form already implemented, the QR method itself is actually pretty simple to implement. I will also devise some tests to verify the correctness of the implementation, probably using examples from the textbook and some random matrices, and I will evaluate the convergence rate of the method for these examples.

1 Introduction

Computing the eigenvalues and eigenvectors of a given $n \times n$ matrix A is a very important process. For example, we saw in our class that the condition number of a symmetric, positive-definite matrix is closely related to its eigenvalues. A naive approach to computing eigenvalues and eigenvectors would be to use the definition: determine the characteristic polynomial by directly computing $\det(A - I\lambda)$, then find the roots numerically, using, say Newton's method with deflation. This naive approach, however, falls apart when n is even slightly large, as the cost of computing the characteristic polynomial scales terribly in n.

Another idea is to use the fact that $A^n x_0 \to \lambda_1^n x_0 + \mathcal{O}(|\lambda_2|^n)$ as $n \to \infty$, where $|\lambda_1| > |\lambda_2|$ are the two largest-in-magnitude eigenvalues of A. By re-scaling appropriately, one obtains the *power method* (Section 5.2 in our book [1]), which is an iterative method for computing the largest eigenvalue of A and a corresponding eigenvector. This method only produces one eigen-pair, and only if A has a unique, largest, simple (having only one eigenvector) eigenvalue; it can be applied repeatedly with deflation to find the other eigenvalues of A, assuming the deflated matrices all have unique, largest, simple eigenvalues.

Using the easily-proved fact that λ is an eigenvalue of A if and only if $(\lambda_0 - \lambda)^{-1}$ is an eigenvalue of $(A - I\lambda_0)^{-1}$, one can then apply the power method to $(A - I\lambda)^{-1}$. By choosing λ_0 judiciously, one can ensure that $(A - I\lambda_0)^{-1}$ has a unique, largest, simple eigenvalue, then apply the power method to determine the eigenvalue. Back-transforming gives a corresponding eigenvalue λ of A. In this way any eigenvalue of A may be obtained by the right choice of λ_0, with a higher convergence rate if λ_0 is chosen close to λ. This is called the *inverse power method* (Section 5.3 in our book [1]); although it overcomes the main difficulty of the power method, it still requires choosing λ_0 properly, which is not trivial.

Rather than use the naive approach, power iteration, or inverse power iteration, modern linear algebra libraries use a method called the *QR method* (Section 5.5 in our book [1]). The QR method relies on iterated QR decompositions, which (with the right optimizations) can be much faster, more robust, and more stable than the above three methods and converges to a matrix that provides all the eigenvalues and eigenvectors of A in one iterative process, avoiding the need for deflation or repeated iterative processes used in the power methods[1].

2 Theory of the QR Method

2.1 Motivation and the basic algorithm

The core idea behind the QR method is the observation that similarity transformations preserve eigenvalues; that is, if P is an invertible matrix, then $B = PAP^{-1}$ has the same eigenvalues as A, and v is an eigenvector of A if and only if Pv is an eigenvector of B. By choosing P cleverly, it may be easier to find the eigenvalues and eigenvectors of B; then the eigenvalues of A are known immediately, and the eigenvectors can be found by applying P^{-1} to the eigenvectors of B, if desired.

Theorem 1. *If P is an invertible matrix, then $B = PAP^{-1}$ has the same eigenvalues as A, and v is an eigenvector of A if and only if Pv is an eigenvector of B.*

Proof. Let λ be an eigenvalue of A with corresponding eigenvector v. Then

$$BPv = PAP^{-1}Pv = PAv = P\lambda v = \lambda Pv,$$

so λ is an eigenvalue of B, and Pv is an eigenvector of B (note that $Pv \neq 0$ because $v \neq 0$ and P is invertible).

[1] Power and inverse power iteration are still useful techniques in some circumstances; for example, to compute only the largest/smallest eigenvalues of a symmetric positive-definite matrix in order to compute the condition number of the matrix.

Conversely suppose that λ is an eigenvalue of B with corresponding eigenvector w. Then

$$A(P^{-1}w) = P^{-1}PAP^{-1}w = P^{-1}Bw = P^{-1}\lambda w = \lambda P^{-1}w,$$

so λ is an eigenvalue of A, and $P^{-1}w$ is an eigenvector of A (again, $P^{-1} \neq 0$ because P is invertible and $w \neq 0$). □

The basic QR method

The main question now is how to choose P. The idea of the QR method is to choose P to be a product of unitary[2] matrices obtained through QR decompositions, with the hope of transforming A through similarity transformations into and triangular matrix B, whose eigenvalues and eigenvectors are easy to compute—note that the upper-triangular requirement seems to fit nicely with the upper triangular matrices that will also be obtained from QR decomposition, and the fact that P needs to be inverted is trivial since we will be constructing it to be unitary. The use of unitary similarity transforms has the nice benefit that it makes the algorithm as a whole more numerically stable than power iteration.

The fact that every matrix is unitarily similar to an upper-triangular matrix by the Schur decomposition theorem—see section 7.2 in Meyer's book [2]—gives us some hope that the above procedure is possible. In fact, the goal of the QR method is essentially to find an (approximate) Schur decomposition of A.

How, then, do we choose P? The simplest idea is just to choose $P = Q$. Then we obtain the following transformation:

$$A \mapsto QAQ^{-1} = QQRQ^*,$$

as $A = QR$. This suggests that $P = Q^{-1} = Q^*$ might work better, as it will cause a cancellation. Then we obtain the basic similarity transformation underlying the QR method:

$$A \mapsto Q^*AQ = Q^*QRQ = RQ. \tag{1}$$

Iterating this transformation yields the basic QR method algorithm, which is described precisely in Algorithm 1. Note that a key requirement for this algorithm to work is that the eigenvalues be *separated*; that is, if $\lambda_1, \ldots, \lambda_n$ are the eigenvalues of A, then

$$|\lambda_1| < |\lambda_2| < \cdots < |\lambda_n|.$$

The basic algorithm may not work without this assumption, as we will see in our numerical experiments and the following theoretical investigation. We will derive the convergence criterion used to stop Algorithm 1 from the convergence theory of the basic method, to which we now turn.

Remark 1. Separation of eigenvalues implies that A is diagonalizable (by Jordan decomposition and the uniqueness of the eigenvalues; see section 7.8 of Meyer's book [2]); thus each eigenvalue has a distinct eigenvector, and the eigenvectors are linearly independent.

Remark 2. The basic QR method is often considered to be most suitable for symmetric positive-definite matrices [1], as the separation of eigenvalues is easier to guarantee, and real arithmetic can be used instead of complex (if the matrix is real) because the eigenvalues and eigenvectors are all real.

[2]One can also consider orthogonal matrices if A is a real matrix, but we begin by focusing on the complex case, as it easier to explain.

Algorithm 1: The Basic QR Method

Input: $A \in \mathbb{C}^{n \times n}$, a matrix with separated eigenvalues
Output: $\lambda_1, \lambda_2, \ldots, \lambda_n \in \mathbb{C}$, the eigenvalues of A
Output: $v_1, v_2, \ldots, v_n \in \mathbb{C}^n$, the eigenvectors of A corresponding to $\lambda_1, \ldots, \lambda_n$

```
1 k ← 0;
2 A_0 ← A;
3 repeat
4 |   Q_k, R_k ← qr(A_k) ;                    // qr is QR decomposition
5 |   A_{k+1} ← R_k Q_k;
6 |   k ← k + 1;
7 until;
```

Convergence of the basic QR method

2.2 The QR method with origin shifts

3 Implementing the QR Method in MATLAB

3.1 Implementing the basic algorithm

3.2 Implementing the algorithm with origin shifts

4 Experiments

References

[1] Ackleh, A. S., Allen, E. J., Kearfott, R. B., and Seshaiyer, P. *Classical and Modern Numerical Analysis*. Chapman and Hall/CRC, 2009.

[2] Meyer, C. D. *Matrix Analysis and Applied Linear Algebra*. Society for Industrial and Applied Mathematics, Philadelphia, 2008.